3. Linux: Basic to Advanced

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 3 of 18 · Thirupathi Keloth · v3.0

Contents

LEVEL 1: BASICS

3.1 What Linux is

3.2 The terminal and the prompt

3.3 Commands, options and arguments

3.4 Where you are: paths and navigation

3.5 The filesystem layout

3.6 Creating, copying, moving and deleting

3.7 Viewing files

3.8 Editing files

3.9 Reading the long listing

3.10 Users, groups and sudo (basics)

3.11 Redirection and pipes (first look)

3.12 Permissions (basics)

3.13 Checking your work

LEVEL 2: OPERATIONS

3.14 Finding things

3.15 Searching inside files: grep

3.16 Text processing: cut, sort, uniq, sed, awk

3.17 Pipes and xargs

3.18 Archives and compression

3.19 Processes and jobs

3.20 Packages (Ubuntu/Debian and RHEL/Amazon Linux)

3.21 Services with systemd

3.22 Logs with journalctl

3.23 Disk space

3.24 Network basics from the shell

3.25 Remote work: SSH, scp and rsync

3.26 Scheduling with cron

3.27 Users and groups in depth

3.28 Advanced permissions

3.29 Environment and shell configuration

LEVEL 3: ADVANCED

3.30 Bash scripting

3.31 How Linux runs programs: processes, memory and the kernel

3.32 Control groups (cgroups) and namespaces: how containers work

3.33 Storage internals: disks, partitions, LVM, RAID

3.34 Performance engineering

3.35 Firewalls and packet filtering: nftables

3.36 Security: capabilities, SELinux/AppArmor, auditing

3.37 Scheduling and timers with systemd

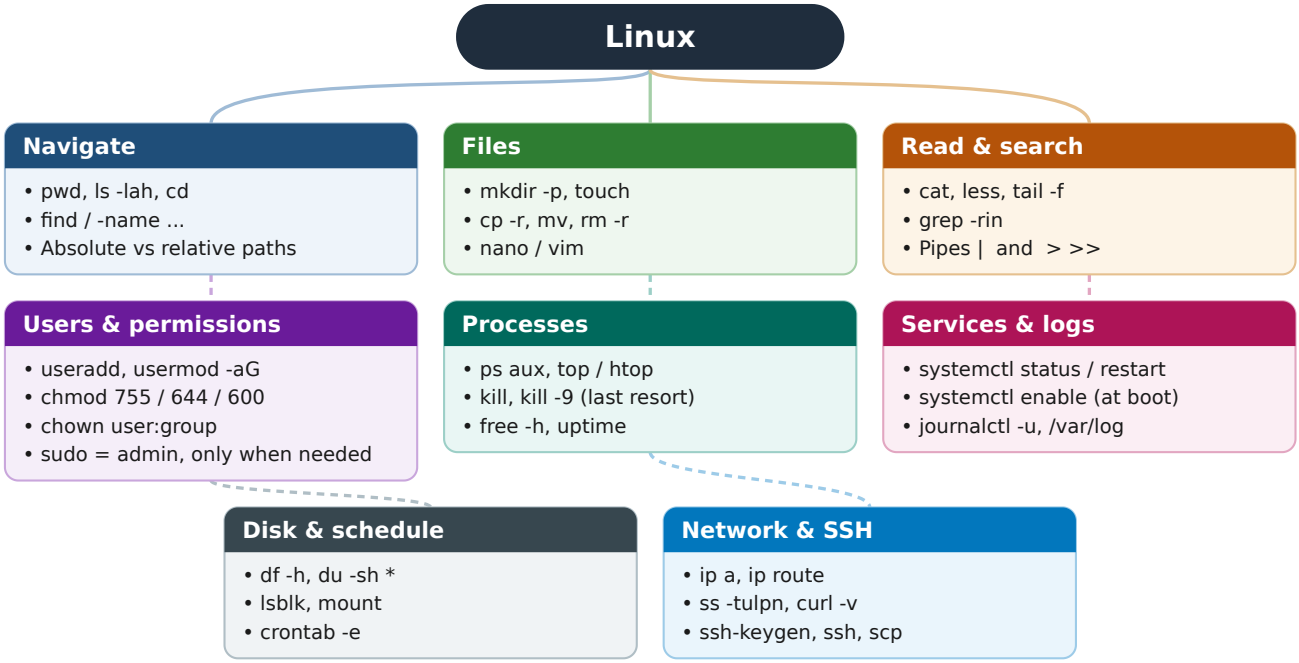
3.38 Logs at scale

3.39 Troubleshooting playbooks

3.40 Hands-on labs (WSL or a VM)

3.41 Interview questions

3.42 Self-check



How to read this book: Level 1 is the foundation, Level 2 is daily operations, Level 3 is the internals and engineering work that senior DevOps roles need. Each level builds on the one before it. Every command shows what it does, the useful options, the expected output, and how to check it worked.

Run the examples in **WSL 2 (Ubuntu)** on your Windows PC. Install with `wsl --install -d Ubuntu` in PowerShell (as Administrator), then restart. Open "Ubuntu" from the Start menu.

LEVEL 1: BASICS

3.1 What Linux is

Linux is an operating system kernel: the core that manages the CPU, memory, disks and network. A **distribution** (distro) adds the tools around that kernel. Ubuntu, Debian, RHEL, Rocky, Amazon Linux and Alpine are distros.

Why it matters for DevOps: most servers, cloud VMs, Kubernetes nodes and containers run Linux. The commands in this book work on all of them.

Term	Meaning
Kernel	The core: it talks to hardware and runs processes
Shell	The program that reads your commands (bash is the most common)
Distribution	Kernel plus packages and tools (Ubuntu, RHEL)
Root	The administrator account, user ID 0
Superuser	Anyone allowed to act as root, usually with `sudo`

3.2 The terminal and the prompt

```
user@DESKTOP:~$
```

Part	Meaning
`user`	Your account
`@DESKTOP`	The machine name
`~`	Your home folder (`/home/user`)
`$`	You're a normal user. Root's prompt ends with `#`

Key shortcuts: `Tab` completes names, `Up` and `Down` recall earlier commands, `Ctrl+C` stops the running command, `Ctrl+L` clears the screen, `Ctrl+R` searches your history.

3.3 Commands, options and arguments

```
ls -l /etc
^^ ^^ ^^^^
|  |   argument: what to act on
|  option: changes behavior (short -l, or long --long)
command
```

Options can be combined: `ls -la` equals `ls -l -a`. Get help with:

```
man ls          # full manual, press q to quit
ls --help       # short summary
type cd         # is it a builtin or a program?
```

3.4 Where you are: paths and navigation

```
pwd             # print working directory
cd /etc         # absolute path: starts at /
cd Documents    # relative path: from the current folder
cd ..           # one level up
cd ~            # home folder
cd -            # back to the previous folder
ls              # list the current folder
ls -la          # long format, including hidden files (names starting with .)
```

Absolute vs relative: `/home/user/notes` always means the same place. `notes` depends on where you are. Scripts use absolute paths.

3.5 The filesystem layout

```
/                root of everything (there is no C:)
├── bin, sbin    essential programs
├── etc          configuration files (/etc/ssh/sshd_config)
├── home         user folders (/home/user)
├── root         the root user's home
├── var          changing data: logs (/var/log), databases, caches
├── tmp          temporary files (cleared at reboot)
├── usr          installed programs and libraries (/usr/bin)
├── opt          optional third-party software
├── dev          device files (/dev/sda, /dev/null)
├── proc, sys    live kernel information (virtual, not on disk)
└── mnt, media   mount points for other disks
```

Windows mapping: `C:\Users\kelot` is like `/home/kelot`. Your Windows drives appear under `/mnt/c` in WSL.

3.6 Creating, copying, moving and deleting

```
mkdir projects                # create a folder
mkdir -p projects/2026/oct    # create nested folders (-p = parents)
touch notes.txt               # create an empty file (or update its timestamp)
cp notes.txt backup.txt       # copy a file
cp -r projects backup/        # copy a folder recursively (-r)
mv notes.txt docs/            # move a file
mv old.txt new.txt            # rename (move to a new name)
rm backup.txt                 # delete a file (no recycle bin!)
rm -r old-folder              # delete a folder and its contents
rmdir empty-folder            # delete only an empty folder
```

Warning: `rm` deletes permanently. `rm -rf /` or `rm -rf ~` is a classic disaster. Always check the path first with `pwd` and `ls`.

Check the result with `ls -l` after each change.

3.7 Viewing files

```
cat file.txt          # print the whole file
less file.txt         # scroll: Space = next page, / = search, q = quit
head -n 5 file.txt    # first 5 lines
tail -n 20 file.txt   # last 20 lines
tail -f /var/log/syslog   # follow a file as it grows (live logs), Ctrl+C to stop
wc -l file.txt        # count lines; -w words; -c bytes
file photo.jpg        # detect the type of a file
```

3.8 Editing files

Editor	When to use	Basic use
`nano`	Beginners, quick edits	`nano file`. Ctrl+O save, Ctrl+X exit
`vim`	Everywhere (servers always have it)	`i` insert, `Esc` normal mode, `:wq` save and quit, `:q!` quit without saving

Start with nano. Learn vim's five essentials (`i`, `Esc`, `/search`, `:wq`, `:q!`) because you'll meet it on every server.

3.9 Reading the long listing

```
ls -l
-rw-r--r-- 1 user user 2950 Oct 22 12:17 .bash_history
drwxr-xr-x 2 user user 4096 Oct 22 13:40 projects
```

Field	Meaning
`-` / `d`	File / directory (also `l` = symbolic link)
`rw-r--r--`	Permissions: owner, group, others (see 3.12)

Field	Meaning
`1`	Number of hard links
`user user`	Owner and group
`2950`	Size in bytes
Date and name	Last modified time and the name

3.10 Users, groups and sudo (basics)

```
whoami          # your user name
id              # your user ID, group ID and all groups
groups          # groups you belong to
sudo apt update # run one command as root (asks for your password)
sudo -l         # what sudo allows you
```

Use `sudo` for a single command. Don't work as root all day: a typo as root can break the whole system.

3.11 Redirection and pipes (first look)

Symbol	Meaning	Example
`>`	Write output to a file (overwrite)	`ls > list.txt`
`>>`	Append output to a file	`echo "line" >> notes.txt`
`<`	Read input from a file	`wc -l < notes.txt`
`|`	Send output of one command to the next	`ls -l | less`
`2>`	Redirect errors	`ls /root 2> errors.txt`
`&>`	Redirect both output and errors	`make &> build.log`

The pipe is the most important idea in Linux: small tools, each doing one job, combined:

```
ls /usr/bin | wc -l        # how many programs are in /usr/bin
```

Same behavior as PowerShell's `>` and `>>` that you used on Day 1, with the same meaning.

3.12 Permissions (basics)

Every file has nine permission bits, three for the owner, three for the group, three for others:

Symbol	Number	Meaning on a file	Meaning on a folder
`r`	4	Read the content	List the names
`w`	2	Change the content	Create, delete, rename entries
`x`	1	Run it as a program	Enter the folder (`cd`)

```
rwxr-x---    = owner rwx (7), group r-x (5), others --- (0)  -> 750
```

```
chmod 644 notes.txt      # owner read/write, group and others read
chmod 700 private/       # only the owner may enter
chmod +x script.sh       # make a script executable
chown user:devs app.log  # change owner and group
ls -l
```

Rule of thumb: `600` for private files (passwords, keys), `644` for normal files, `755` for programs and folders others need to enter. Never use `777`.

3.13 Checking your work

Each section says what to expect. The habit that prevents most mistakes is:

1. Know where you are: `pwd`
2. Look before you act: `ls -la`
3. Act: the command
4. Verify: `ls`, `cat`, or the command's own status

Exit status: every command returns a number. `0` means success, anything else means failure. Check the last one with `echo $?`:

```
ls /etc      ; echo $?    # prints 0
ls /nope     ; echo $?    # prints 2 (error)
```

Level 1 self-check

- [ ] I can move around with `cd`, `pwd`, `ls`, and use absolute and relative paths.
- [ ] I can create, copy, move and delete files and folders safely.
- [ ] I can read files with `cat`, `less`, `head`, `tail -f`.
- [ ] I can read `ls -l` and explain the permission string.
- [ ] I can use `>`, `>>` and `|` together.
- [ ] I can check a command's exit status with `echo $?`.

LEVEL 2: OPERATIONS

3.14 Finding things

```
find /etc -name "*.conf"                # by name
find . -type f -size +100M              # files over 100 MB in this tree
find /var/log -mtime -1                 # changed in the last 24 hours
find . -name "*.log" -exec ls -lh {} \;  # run a command on each result
find . -name "*.tmp" -delete            # careful: deletes matches
locate nginx.conf                       # fast index search (run sudo updatedb first)
which python3                           # where the program lives in $PATH
```

`find` walks the tree live (always current, slower). `locate` uses an index (fast, may be stale).

3.15 Searching inside files: grep

```
grep "error" app.log                # lines containing "error"
grep -i "error" app.log             # case-insensitive
grep -n "error" app.log             # show line numbers
grep -v "debug" app.log             # invert: lines WITHOUT "debug"
grep -r "DB_HOST" /etc/app/         # recursive search of a folder
grep -E "(warn|error)[0-9]+" app.log   # extended regular expressions
grep -c "ERROR" app.log             # count matching lines
grep -A2 -B2 "timeout" app.log      # 2 lines of context after and before
```

Regular expressions basics:

Pattern	Matches
`.`	any one character
`*`	zero or more of the previous
`^` / `$`	start / end of line
`[0-9]`	any digit
`\.`	a literal dot

3.16 Text processing: cut, sort, uniq, sed, awk

```
cut -d: -f1 /etc/passwd            # first field, colon-separated: user names
sort names.txt                     # alphabetical sort
sort -n numbers.txt                # numeric sort
sort access.log | uniq -c          # count duplicate lines (uniq needs sorted input)
sort -rn counts.txt | head -5      # top 5, highest first
```

sed edits streams of text:

```
sed 's/localhost/10.0.0.5/' app.conf          # replace the first match on each line
sed 's/old/new/g' file.txt                    # replace all matches (g = global)
sed -i 's/debug/info/g' app.conf              # edit the file in place (-i)
sed -n '10,20p' file.txt                      # print lines 10 to 20
sed '/^#/d' app.conf                          # delete comment lines
```

awk works on columns (fields) of each line:

```
awk '{print $1}' access.log                   # first column (IP address)
awk -F: '{print $1, $3}' /etc/passwd          # user name and user ID
awk '$9 == 404 {print $7}' access.log         # URLs with status 404 (nginx format)
awk '{sum += $3} END {print sum}' sizes.txt   # total of column 3
```

Pipelines combine them. Count the top 10 client IPs in an access log:

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

Read it left to right: take the IP column, sort, count duplicates, sort by count descending, keep ten.

3.17 Pipes and xargs

```
ls *.log | wc -l                          # how many log files
find . -name "*.tmp" | xargs rm -v        # delete files named in a list
echo "a b c" | xargs -n1 echo             # run echo once per word
```

`xargs` turns standard input into arguments for another command. Use `find ... -print0 | xargs -0` for file names with spaces.

3.18 Archives and compression

```
tar -czf backup.tar.gz projects/       # create a compressed archive (c=create z=gzip f=file)
tar -tzf backup.tar.gz                 # list contents (t=list)
tar -xzf backup.tar.gz -C /tmp/restore # extract into a folder (x=extract)
gzip big.log                           # compress (replaces the file with big.log.gz)
gunzip big.log.gz                      # decompress
zcat big.log.gz | grep ERROR           # search a compressed log without extracting
```

3.19 Processes and jobs

A **process** is a running program. Each has a PID (process ID).

```
ps aux                 # all processes: user, PID, CPU%, MEM%, command
ps -ef --forest        # process tree
top                    # live view. Press q to quit; P sorts by CPU, M by memory
htop                   # friendlier top (install: sudo apt install htop)
pgrep -a nginx         # PIDs of processes matching a name
pstree -p              # tree with PIDs
```

Background and foreground:

```
sleep 300 &            # run in background (& at the end)
jobs                   # list background jobs
fg %1                  # bring job 1 to foreground
Ctrl+Z                 # suspend the running program
bg %1                  # continue it in the background
nohup ./long.sh &      # keep running after you log out
```

Signals stop or control processes:

Signal	Number	Meaning
SIGTERM	15	Politely ask to stop (default for `kill`)
SIGKILL	9	Force stop; cannot be caught. Last resort
SIGHUP	1	Reload configuration (many daemons)
SIGSTOP / SIGCONT	19 / 18	Pause and resume

```
kill 4321              # SIGTERM to PID 4321
kill -9 4321           # SIGKILL: only if SIGTERM fails
pkill -f "python app.py"  # kill by name pattern
kill -HUP $(pgrep nginx)  # reload nginx config
```

Always try SIGTERM first. A process that receives SIGKILL cannot clean up its files.

3.20 Packages (Ubuntu/Debian and RHEL/Amazon Linux)

Task	Debian/Ubuntu (apt)	RHEL/Amazon Linux (dnf)
Update package lists	`sudo apt update`	`sudo dnf check-update`
Upgrade installed packages	`sudo apt upgrade`	`sudo dnf upgrade`
Install	`sudo apt install nginx`	`sudo dnf install nginx`
Remove	`sudo apt remove nginx`	`sudo dnf remove nginx`
Search	`apt search nginx`	`dnf search nginx`
Show info	`apt show nginx`	`dnf info nginx`
Which package owns a file	`dpkg -S /usr/sbin/nginx`	`rpm -qf /usr/sbin/nginx`

Verify an install:

```
nginx -v                # prints the version
apt list --installed | grep nginx
```

3.21 Services with systemd

`systemd` starts, stops and supervises services. It's PID 1 on modern Linux.

```
systemctl status nginx          # state, PID, last log lines
sudo systemctl start nginx      # start now
sudo systemctl stop nginx       # stop now
sudo systemctl restart nginx    # stop, then start
sudo systemctl reload nginx     # reload config without dropping connections
sudo systemctl enable nginx     # start automatically at boot
sudo systemctl disable nginx    # don't start at boot
systemctl is-active nginx       # prints active or inactive
systemctl list-units --failed   # services that crashed
systemctl cat nginx             # the unit file in use
```

Expected output from `systemctl is-active nginx`: `active`. If it prints `failed`, read the logs (3.22).

3.22 Logs with journalctl

```
journalctl                      # all logs, oldest first
journalctl -u nginx             # one service
journalctl -u nginx -f          # follow live
journalctl -u nginx --since "10 min ago"
journalctl -p err -b            # errors since the last boot (-p = priority)
journalctl -k                   # kernel messages (like dmesg)
journalctl --disk-usage         # space used by logs
sudo journalctl --vacuum-size=500M   # shrink logs to 500 MB
```

Traditional log files live in `/var/log` (`syslog`, `auth.log`, `nginx/error.log`). Read with `less`, `tail -f`, `grep`.

3.23 Disk space

```
df -h                    # space per filesystem (human-readable)
df -i                    # inodes used (you can run out with free space left)
du -sh projects/         # total size of one folder
du -xh --max-depth=1 / 2>/dev/null | sort -h | tail   # biggest top-level folders
lsblk                    # block devices and mount points (disks, partitions)
mount | grep sda         # what is mounted where
```

Classic trap: a file was deleted but `df` still shows the space used. Some process still holds the file open:

```
sudo lsof +L1           # open files whose link count is 0 (deleted)
```

Restart that process, or truncate instead of deleting: `: > big.log`.

3.24 Network basics from the shell

```
ip addr                      # interfaces and IP addresses
ip route                     # routing table, default gateway
ip neigh                     # ARP table (IP to MAC)
ss -tulpn                    # listening ports and the program using each
ss -tan state established    # current TCP connections
ping -c 4 8.8.8.8            # reachability by IP (-c = count)
traceroute -n 8.8.8.8        # path (apt install traceroute)
dig +short example.com       # DNS lookup (apt install dnsutils)
curl -I https://example.com  # HTTP headers only
curl -s -o /dev/null -w "%{http_code} %{time_total}s\n" https://example.com
nc -zv 10.0.0.5 5432         # is a TCP port open? (-z scan only, -v verbose)
```

The `ss` columns to read: `State`, `Local Address:Port`, `Peer Address:Port`, `Process`.

3.25 Remote work: SSH, scp and rsync

```
ssh-keygen -t ed25519 -C "kelot@omen"   # create a key pair (press Enter for defaults)
ssh-copy-id user@10.0.0.5               # install your public key on a server
ssh user@10.0.0.5                       # log in with the key, no password
ssh -i ~/.ssh/key.pem ec2-user@1.2.3.4  # use a specific key file
scp notes.txt user@10.0.0.5:/tmp/       # copy a file to a server
scp -r user@10.0.0.5:/var/log/app ./    # copy a folder from a server
rsync -avz --delete ./site/ user@10.0.0.5:/var/www/site/   # sync (only changes move)
```

Key files: private key `~/.ssh/id_ed25519` (never share, never commit), public key `~/.ssh/id_ed25519.pub` (safe to share). The server's `~/.ssh/authorized_keys` holds the public keys allowed in.

Permissions SSH enforces: `~/.ssh` must be `700`, private keys `600`. Otherwise SSH refuses to use them.

3.26 Scheduling with cron

```
crontab -e          # edit your cron jobs
crontab -l          # list them
```

Format: `minute hour day-of-month month day-of-week command`

```
30 2 * * *        /home/user/backup.sh >> /var/log/backup.log 2>&1   # 02:30 every day
*/5 * * * *       /home/user/check.sh                                # every 5 minutes
0 9 * * 1-5       /home/user/report.sh                               # 09:00 Monday to Friday
@reboot           /home/user/start.sh                                # at boot
```

Rules that prevent most cron failures: use absolute paths (cron's `PATH` is minimal), redirect output to a log, and test the script by running it by hand first.

3.27 Users and groups in depth

```
sudo useradd -m -s /bin/bash alice     # create user with a home folder and bash shell
sudo passwd alice                      # set password
sudo usermod -aG docker alice          # add to group docker (-a = append!)
sudo userdel -r alice                  # delete user and home folder
getent passwd alice                    # user record
getent group docker                    # group record and members
```

Files that store accounts: `/etc/passwd` (accounts), `/etc/shadow` (password hashes, root only), `/etc/group` (groups), `/etc/sudoers` (edit only with `sudo visudo`).

Warning: `usermod -G docker alice` without `-a` removes alice from every other group. Always use `-aG`.

3.28 Advanced permissions

Special bits:

Bit	Number	On a file	On a folder
setuid	4	Runs as the file's owner (`passwd` uses this)	No effect on Linux
setgid	2	Runs as the file's group	New files inherit the folder's group (shared team folders)
sticky	1	No effect	Only the file owner can delete (`/tmp` uses this)

```
chmod 1777 /shared     # sticky + world-writable folder
chmod 2775 /team       # setgid: shared group folder
ls -l /usr/bin/passwd  # -rwsr-xr-x : the s is setuid
```

Access Control Lists (ACLs) give per-user rules beyond the three owner, group and other classes:

```
setfacl -m u:alice:rx /srv/app        # give alice read and enter
getfacl /srv/app                      # view the rules
setfacl -R -m g:devs:rwX /srv/shared  # recursive, X = execute only on folders
```

A `+` at the end of `ls -l` output means an ACL exists.

umask sets the default permissions for new files. `umask 022` makes new files `644` and new folders `755`.

3.29 Environment and shell configuration

```
echo $HOME            # value of a variable
echo $PATH            # list of folders searched for commands
export APP_ENV=dev    # set a variable for this shell and its children
env | grep APP        # show environment variables
```

Configuration files:

File	Read by	Use
`~/.bashrc`	Every new interactive shell	Aliases, prompt, functions
`~/.profile` / `~/.bash_profile`	Login shells	Environment variables, PATH
`/etc/environment`	All users	System-wide variables

Example additions to `~/.bashrc`:

```
alias ll='ls -lah'
export PATH="$HOME/bin:$PATH"
```

Apply without reopening: `source ~/.bashrc`. Use `type ll` to verify the alias.

Level 2 self-check

- [ ] I can find files by name, size and age, and search text with `grep` (including `-r`, `-v`, `-E`).
- [ ] I can build pipelines with `sort`, `uniq -c`, `awk` and `sed`.
- [ ] I can kill a process correctly (SIGTERM first) and explain signals.
- [ ] I can manage services with `systemctl` and read logs with `journalctl`.
- [ ] I can diagnose disk, network and port problems from the shell.
- [ ] I can set up SSH keys, copy files with `scp` and `rsync`, and write a cron job.
- [ ] I can set ACLs and explain setuid, setgid and sticky bits.

LEVEL 3: ADVANCED

3.30 Bash scripting

A script that is safe to run

```
#!/usr/bin/env bash
# backup.sh: archive a folder with a date stamp, keep the last 7
set -euo pipefail          # exit on error, on unset variables, on pipe failures

readonly SRC="${1:?usage: $0 <source-dir>}"
readonly DEST="/var/backups/app"
readonly STAMP="$(date +%Y%m%d-%H%M%S)"

log() { printf '%s %s\n' "$(date +%T)" "$*" >&2; }

cleanup() { log "finished (exit $?)"; }
trap cleanup EXIT          # always runs, even on failure

mkdir -p "$DEST"
log "archiving $SRC"
tar -czf "$DEST/app-$STAMP.tar.gz" -C "$(dirname "$SRC")" "$(basename "$SRC")"

# keep only the 7 newest archives
ls -1t "$DEST"/app-*.tar.gz | tail -n +8 | xargs -r rm -v --
```

Run it: `chmod +x backup.sh && ./backup.sh /home/user/projects`

What each safety line does:

Line	Effect
`set -e`	Stop at the first failing command (instead of continuing)
`set -u`	Error on an unset variable (catches typos like `$SRCC`)
`set -o pipefail`	A pipeline fails if any part fails
`${1:?message}`	Stop with a message if the argument is missing
`trap ... EXIT`	Run cleanup code however the script ends

Variables, quoting and tests

```
name="my file.txt"
cp "$name" backup/        # always quote variables: spaces would split them
echo "${name%.txt}"       # remove suffix -> my file
echo "${#name}"           # length -> 11
echo "${name:-default}"   # use default if unset or empty

if [[ -f "$name" ]]; then echo "file exists"; fi
[[ -d logs ]] && echo "logs folder exists"
[[ "$ENV" == "prod" ]] || { echo "not prod"; exit 1; }
```

Test operators: `-f` file exists, `-d` folder exists, `-s` file not empty, `-x` executable, `-z` string empty, `-n` string not empty, `-eq -ne -lt -gt` for numbers, `==` and `!=` for strings.

Loops, functions and exit codes

```
for f in /var/log/*.log; do
  [[ -s "$f" ]] || continue            # skip empty files
  echo "$f: $(wc -l < "$f") lines"
done

while read -r host; do
  if ping -c1 -W2 "$host" &>/dev/null; then
    echo "UP   $host"
  else
    echo "DOWN $host" >&2
  fi
done < hosts.txt

check_port() {                         # function
  local host="$1" port="$2"            # local keeps variables inside
  nc -z -w3 "$host" "$port"
}
check_port 10.0.0.5 5432 && echo "db reachable" || echo "db unreachable"
```

Exit codes: a function or script returns its last command's code, or `return N` / `exit N`. `$?` holds the last code. `&&` runs the next command only on success, `||` only on failure.

Shell debugging

```
bash -n script.sh     # syntax check only
bash -x script.sh     # trace each command as it runs
shellcheck script.sh  # static analysis: finds quoting bugs (apt install shellcheck)
```

3.31 How Linux runs programs: processes, memory and the kernel

The process model

Every process has: a PID, a parent (PPID), an owner (UID), open file descriptors (0 stdin, 1 stdout, 2 stderr), and memory. New processes are created with `fork()` (copy the parent) and then `exec()` (replace the copy with a new program). Your shell does exactly this for every command.

```
ps -o pid,ppid,user,stat,cmd -p $$     # your shell: PID, parent, state
ls -l /proc/$$/fd                       # its open file descriptors
cat /proc/$$/status | head -5           # name, state, PID, PPID, UID
```

Process states in `ps` (the STAT column): `R` running, `S` sleeping (waiting, interruptible), `D` uninterruptible sleep (usually waiting for disk: cannot be killed), `T` stopped, `Z` zombie (finished, parent has not collected its exit code).

Zombies are harmless individually; a parent that never calls `wait()` keeps them. The fix is to repair or restart the parent.

/proc and /sys: the kernel as files

`/proc` holds live kernel data as text files:

```
cat /proc/cpuinfo | grep "model name" | head -1
cat /proc/meminfo | head -5
cat /proc/loadavg
cat /proc/<PID>/cmdline | tr '\0' ' '     # full command line
cat /proc/<PID>/limits                    # resource limits (open files, etc.)
ls /proc/<PID>/fd | wc -l                 # open file count
```

`/sys` exposes devices and settings:

```
cat /sys/block/sda/queue/scheduler        # I/O scheduler in use
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward   # enable IP forwarding now (not at boot)
sysctl net.ipv4.ip_forward                # read a kernel parameter
sudo sysctl -w vm.swappiness=10           # change now
echo "vm.swappiness=10" | sudo tee /etc/sysctl.d/99-tuning.conf   # make permanent
sudo sysctl --system                      # load all sysctl files
```

Memory: what "used" really means

```
free -h
#               total        used        free      shared  buff/cache   available
# Mem:           15Gi        4.2Gi       2.1Gi       ...       8.9Gi        10Gi
```

Read the **available** column, not "free". Linux uses spare RAM as page cache (fast disk reads), and gives it back when apps need memory. Low "free" is healthy.

Virtual memory in one picture:

```
process address space          physical RAM
[ code | heap | stack ]  --->  pages (4 KB each)
                               pages not used recently -> swap (disk) if enabled
page fault: process touches a page not in RAM -> kernel loads it
```

Key checks: `vmstat 1 5` (columns `si` / `so` = swap in/out; non-zero for long periods means memory pressure), `swapon --show`.

OOM killer

When memory runs out, the kernel kills a process to survive:

```
dmesg -T | grep -i -E "killed process|out of memory"
journalctl -k | grep -i oom
```

Each process has an OOM score: `cat /proc/<PID>/oom_score`. Protect critical services with systemd: `OOMScoreAdjust=-500` in the unit file. Control memory per service: `MemoryMax=2G`.

3.32 Control groups (cgroups) and namespaces: how containers work

Containers are not a Linux feature by themselves. They are ordinary processes isolated with two kernel mechanisms:

Mechanism	Isolates	Example
Namespaces	What a process can see: PIDs, network, mounts, hostname, users	A container sees its own PID 1 and its own network interface
cgroups	How much a process can use: CPU, memory, I/O, number of processes	A container limited to 512 MB of memory

See your own namespaces

```
ls -l /proc/$$/ns/          # one link per namespace type (pid, net, mnt, uts, user, ...)
readlink /proc/$$/ns/pid    # e.g. pid:[4026531836] : the namespace ID
```

Two processes with the same ID share that namespace.

Build a namespace yourself

```
sudo unshare --pid --fork --mount-proc bash
# inside: ps aux  -> you see only this shell as PID 1
exit
```

`unshare` creates new namespaces and runs a program in them. Docker does the same with more layers.

Limit memory with cgroups v2

```
cat /sys/fs/cgroup/cgroup.controllers       # available controllers: cpu, memory, io, pids
sudo mkdir /sys/fs/cgroup/demo
echo 100M | sudo tee /sys/fs/cgroup/demo/memory.max
echo "$$" | sudo tee /sys/fs/cgroup/demo/cgroup.procs   # move your shell into the group
cat /sys/fs/cgroup/demo/memory.current      # current usage in bytes
```

Now any memory your shell allocates beyond 100 MB is limited by the kernel. systemd exposes the same features: `systemd-run --scope -p MemoryMax=200M bash`.

3.33 Storage internals: disks, partitions, LVM, RAID

Stack from disk to files

```
physical disk (/dev/sda, /dev/nvme0n1)
   -> partition table (GPT): partitions (/dev/sda1)
      -> LVM physical volume (PV) -> volume group (VG) -> logical volumes (LV)
         -> filesystem (ext4, xfs)  -> mount point (/data)
```

```
lsblk -f                      # disks, partitions, filesystems, UUIDs, mount points
sudo fdisk -l /dev/sda        # partition table (GPT/MBR)
sudo blkid                    # filesystem types and UUIDs
```

LVM: resize storage without downtime

```
sudo pvcreate /dev/sdb                    # prepare a disk as a physical volume
sudo vgcreate data_vg /dev/sdb            # create a volume group
sudo lvcreate -L 20G -n app_lv data_vg    # carve a 20 GB logical volume
sudo mkfs.ext4 /dev/data_vg/app_lv        # create a filesystem
sudo mount /dev/data_vg/app_lv /data      # mount it

# later: add a disk and grow the volume online
sudo vgextend data_vg /dev/sdc
sudo lvextend -r -L +10G /data_vg/app_lv  # -r resizes the filesystem too
sudo vgs ; sudo lvs                       # check free space and volumes
```

RAID with mdadm

Level	Redundancy	Capacity	Use
RAID 0	None (fast, one disk failure loses all)	Sum	Scratch data
RAID 1	Mirror	Half	System disks
RAID 5	One disk can fail	$N - 1$	General storage

Level	Redundancy	Capacity	Use
RAID 10	Mirror of stripes	Half	Databases

```
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc
cat /proc/mdstat                # sync status
sudo mdadm --detail /dev/md0    # health
```

RAID is not a backup. It protects against disk failure, not deletion, corruption or ransomware.

I/O performance

```
iostat -xz 1 3          # per device: %util, await (ms latency), r/s, w/s
cat /sys/block/sda/queue/scheduler
sudo hdparm -tT /dev/sda  # raw read speed (SATA disks)
```

Reading `iostat`: `%util` near 100 with high `await` means the disk is the bottleneck. `await` near 0 with high `%util` on SSD can still be normal under parallel load.

3.34 Performance engineering

The USE method (Brendan Gregg)

For every resource (CPU, memory, disk, network), check **Utilization**, **Saturation** (queued work), and **Errors**. Run this checklist when something is slow:

```
uptime                # load averages over 1, 5, 15 minutes
dmesg -T | tail       # kernel errors
vmstat 1 5            # r (run queue), b (blocked), si/so (swap), us/sy/id/wa/st (CPU)
mpstat -P ALL 1 3     # per-CPU: one busy CPU = single-threaded bottleneck
pidstat 1 3           # per-process CPU
iostat -xz 1 3        # disk
free -m               # memory
sar -n DEV 1 3        # network throughput (apt install sysstat)
sar -n TCP,ETCP 1 3   # TCP connections and retransmits
```

Load average: the number of processes running or waiting for CPU or disk. Compare with the CPU count (`nproc`). Load 8 on 4 CPUs means work is queuing.

Tracing system calls and functions

```
strace -p 4321                  # system calls of a live process (Ctrl+C to stop)
strace -c ls /etc               # summary: how many calls and their time
strace -f -e trace=openat ./app  # follow children, only file-open calls
ltrace -p 4321                  # library calls (apt install ltrace)
```

Example reading: `strace` shows `read(3, ...) = 0` repeatedly and `select` timeouts: the app is waiting on something. The timestamps show where time goes.

perf and bpftrace (kernel-level profiling)

```
sudo perf top                          # live CPU hot functions (kernel and user)
sudo perf record -g -p 4321 sleep 30   # sample a process with call graphs for 30 s
sudo perf report                       # interactive report of the recording
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat {
  printf("%s %s\n", comm, str(args->filename)); }'
```

`bpftrace` uses eBPF: small programs the kernel runs safely on events. It answers "which process opens which file, right now" without restarting anything.

Network performance

```
ss -s                     # socket summary
ss -tin                   # per-connection internals: rtt, cwnd, retransmits
nstat -az | grep -i retrans
ethtool eth0              # link speed, duplex (apt install ethtool)
```

3.35 Firewalls and packet filtering: nftables

Linux filters packets in the kernel with **netfilter**. Modern tools: `nftables` (`nft`) and the older `iptables` (now often a wrapper over nftables).

Packet path, simplified:

```
incoming packet -> PREROUTING -> routing decision -> INPUT (to this host) -> local process
                                                  -> FORWARD (via this host) -> POSTROUTING -> out
local process -> OUTPUT -> POSTROUTING -> out
```

A minimal server firewall with nftables:

```
sudo nft list ruleset                  # show current rules
```

```
# /etc/nftables.conf
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept      # replies to our own connections
    iif lo accept                            # local traffic
    ip protocol icmp accept                  # ping
    tcp dport { 22, 80, 443 } ct state new accept   # SSH and web
    counter drop                             # count what is dropped
  }
  chain forward { type filter hook forward priority 0; policy drop; }
  chain output  { type filter hook output priority 0; policy accept; }
}
```

```
sudo nft -c -f /etc/nftables.conf      # check syntax first (-c)
sudo nft -f /etc/nftables.conf         # load it
sudo nft list chain inet filter input  # verify, with packet counters
```

Lockout warning: when you set `policy drop` remotely, first allow your SSH port. Test from a second session before closing the first. Keep a console or cloud serial-console path.

Conntrack shows tracked connections: `sudo conntrack -L | head`.

Routing and NAT

```
ip route add 10.20.0.0/16 via 10.0.0.1 dev eth0    # static route
ip rule show                                       # policy routing rules
sudo sysctl -w net.ipv4.ip_forward=1               # act as a router
sudo nft add table ip nat
sudo nft 'add chain ip nat postrouting { type nat hook postrouting priority 100; }'
sudo nft add rule ip nat postrouting oifname "eth0" masquerade    # NAT for outgoing traffic
```

3.36 Security: capabilities, SELinux/AppArmor, auditing

Least privilege

- **Capabilities** split root's power into about 40 pieces: `CAP_NET_BIND_SERVICE` (bind ports below 1024), `CAP_SYS_ADMIN` (very broad, avoid). Check: `getcap /usr/bin/ping`. Grant only one: `sudo setcap cap_net_bind_service=+ep /usr/local/bin/app`.
- **seccomp** restricts which system calls a process may make (containers use it by default).
- **MAC systems** enforce rules on top of permissions: **SELinux** (RHEL, Amazon Linux) labels every file and process; **AppArmor** (Ubuntu) uses per-program profiles.

```
getenforce                      # SELinux: Enforcing, Permissive or Disabled (RHEL)
ls -Z /var/www/html             # SELinux labels on files
sudo aa-status                  # AppArmor profiles loaded (Ubuntu)
ausearch -m avc -ts recent      # SELinux denials (audit log)
```

When a service works in a shell but fails under systemd or SELinux, check denials before changing permissions.

Auditing and hardening

```
sudo auditctl -w /etc/passwd -p wa -k identity    # log writes/attribute changes to /etc/passwd
sudo ausearch -k identity                         # find those events
last -n 20                                        # recent logins
lastb | head                                      # failed logins
sudo lynis audit system                           # hardening report (apt install lynis)
```

SSH hardening in `/etc/ssh/sshd_config` (then `sudo sshd -t` to test, `sudo systemctl reload ssh`):

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers alice deploy
```

Never disable password login before your key works. Test in a second session.

Other hardening habits: automatic security updates (`unattended-upgrades` on Ubuntu, `dnf-automatic` on RHEL), no shared accounts, sudo logs enabled, file integrity monitoring (AIDE), and secrets outside `/etc` config files readable by others.

3.37 Scheduling and timers with systemd

systemd timers replace cron where you need dependencies, logging and missed-run handling:

```
# /etc/systemd/system/backup.service
[Unit]
Description=Nightly project backup
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh /home/user/projects
User=user
```

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup nightly
[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true              # run at next boot if the machine was off
[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now backup.timer
systemctl list-timers              # next and last run times
journalctl -u backup.service       # output of the job
```

For a long-running service, write a unit with `Restart=on-failure`, `RestartSec=5`, and resource limits such as `MemoryMax=`. Check its configuration with `systemd-analyze verify`.

3.38 Logs at scale

- Structured logs: JSON lines are easy to parse: `jq -r '.level' app.log | sort | uniq -c`.
- Rotation: `logrotate` (`/etc/logrotate.d/app`): daily, keep 14, compress.
- Forwarding: `rsyslog`, `journald` with `ForwardToSyslog`, or agents (Fluent Bit, Vector, CloudWatch agent) to a central store.
- Time: all servers must agree. Check with `timedatectl` and `chronyc tracking` (chrony is the modern NTP client). Wrong clocks break TLS, Kerberos and log correlation.

3.39 Troubleshooting playbooks

A. Server is slow 1. `uptime`, `vmstat 1 5`, `top`: what is busy? CPU, memory, I/O? 2. CPU busy: `pidstat 1 3`, then `perf top` for the hot function. 3. Memory: `free -h` (use available), check swap (`si` / `so`), find the big process, check OOM logs. 4. Disk: `iostat -xz 1 3`, find the process with `iotop` (apt install iotop) or `pidstat -d 1`. 5. Network: `sar -n TCP,ETCP 1 3` for retransmits; `ss -s` for socket counts.

B. Service is down 1. `systemctl status svc` and `journalctl -u svc -n 100`. 2. Config test: `nginx -t`, `sshd -t`, the app's own check command. 3. Port conflict: `ss -tulpn | grep :80`. 4. Permissions: `namei -l /path/to/file`, then SELinux denials (`ausearch -m avc`). 5. Dependencies: database reachable? DNS works? (`dig`, `nc -zv`).

C. Disk full 1. `df -h` and `df -i` (inodes). 2. `du -xh --max-depth=1 / | sort -h | tail` to find the folder. 3. Deleted but open: `sudo lsof +L1`. 4. Fix: rotate or delete old logs, restart the holder, or grow the LVM volume (3.33).

D. Cannot reach a host 1. Local? `ip addr`, `ip route`. 2. Name? `dig +short host`, then `dig +short host @8.8.8.8`. 3. Port? `nc -zv host 443`. Refused means nothing listens. Timeout means a firewall or route. 4. Trace: `traceroute -n host`, and `sudo nft list ruleset` on both ends. 5. Capture: `sudo tcpdump -i any host 10.0.0.5 and port 443 -nn` (apt install tcpdump).

E. Process will not die 1. `ps -o pid,stat,cmd -p PID`. State `D` means waiting on I/O: wait or fix the storage. State `Z` means a zombie: restart its parent. 2. `kill PID`, wait, then `kill -9 PID` only if needed.

3.40 Hands-on labs (WSL or a VM)

#	Lab	What you prove
1	Build a folder tree with `mkdir -p`, copy and move files, then undo it all with `rm -r`	Level 1 navigation and safety
2	Write a pipeline that counts the top five commands in your `~/.bash_history`	Pipes, `sort`, `uniq`, `head`
3	Create a 100 MB file, find it with `find`, compress it, and compare sizes with `du`	Find, tar, du
4	Install nginx, use `systemctl`, break its config on purpose, read the error with `nginx -t` and `journalctl`	Services and logs
5	Write `backup.sh` from 3.30 with `set -euo pipefail` and a trap, then run `shellcheck` and fix every warning	Scripting quality
6	Run `unshare --pid --fork --mount-proc bash`, confirm PID 1, then create a cgroup with a 100 MB memory limit and watch it apply	Namespaces and cgroups
7	Create a loop device as a fake disk, put it in LVM, grow the volume online	Storage stack
8	Run `stress-ng --cpu 4 --timeout 60s` and diagnose it with `vmstat`, `mpstat`, `pidstat` (apt install stress-ng)	Performance tools
9	Write an nftables ruleset that allows only SSH and HTTP, test with `nft -c`, and verify with counters	Firewalls
10	Create a systemd service and timer that writes the date to a log every five minutes, then check `journalctl`	systemd automation
11	Run the playbook in 3.39 on a fault you create: fill a disk with a deleted-open file and fix it	Troubleshooting

3.41 Interview questions

Basics 1. What is the difference between `rm` and `rmdir`, and why is `rm -rf` dangerous? 2. Explain `chmod 750`. What does `setgid` do on a folder? 3. How does a pipe differ from redirection?

Intermediate 4. How do you find the process holding port 8080, and how do you stop it safely? 5. A disk shows full, but `du` finds little. Why? 6. What is the difference between `systemctl restart` and `reload`? 7. Why should you never run `usermod -G` without `-a`?

Advanced 8. What are namespaces and cgroups? How do containers use them? 9. What does `available` mean in `free -h`, and why is low "free" normal? 10. Explain the USE method and apply it to a slow server. 11. What is an OOM kill, how do you find it, and how do you protect a critical service? 12. Describe the storage stack from disk to mount point with LVM. 13. How would you lock down an SSH server without losing access? 14. What is the difference between `iptables` and `nftables`, and how does netfilter process a packet? 15. A process is stuck in state `D`. What does that mean and what do you do?

3.42 Self-check

- [ ] Level 1: I can navigate, manage files and read permissions without help.
- [ ] Level 2: I can search, transform text, manage services and logs, and diagnose disk, network and port problems.
- [ ] Level 3: I can write a safe Bash script, explain processes and memory, describe how containers use namespaces and cgroups, manage LVM, and use the performance and tracing tools.

- [ ] I can write a firewall, harden SSH safely, and explain SELinux or AppArmor denials.
- [ ] I can work through each troubleshooting playbook and explain the result to someone else.

REMEMBER

- Permissions: **r = 4, w = 2, x = 1** → 7 = rwx, 6 = rw-, 5 = r-x, 4 = r--. Private keys: **600**.
- Cron fields in order: **minute hour day month weekday** ("**M**y **H**amster **D**oesn't **M**ind **W**aiting").
- Service broken? **status → logs → resources → network → config**.
- Linux has no recycle bin: double-check before `rm -rf`.